Roshni, set up for you: the personalized learning plan

Roshni remembers how you learn, opens every page the way you need it, and always knows your next step. Plain code decides what to show you; AI does the heavy work your needs call for, and Fact Guard checks it.

Roshni · AICON'26 Build With AI · 404 Team Not Found · Plan written Saturday 10 October 2026

6 features · 7 h	**2 features · 1 h**	**5 features · 17 h**
Today, before submission	Demo polish, if time allows	After the hackathon

1. Design principles

- Behaviour over questionnaires. Tailor from what the learner actually does (levels chosen, quiz results, tabs opened). Setup asks at most two questions, and every answer changes something the learner can see immediately.
- Personalize the frame, not the facts. Personalization chooses WHICH checked version to show (tab, level, language, weak sections); it never rewrites content per person. The cache, demo mode and Fact Guard stay intact.
- Every personal sentence must come from stored data or a number on screen. If the data isn't there, the sentence isn't shown: no empty 'Keep going!' filler.
- Explain every tailored default in one line ('Opened in Listen because you said you use a screen reader. Change') and keep every control one tap away. Defaults, never locks.
- Label code as code and AI as AI. The study plan and next step are plain code and say so; the rewrite, word explanations, questions and voice answers are AI and are checked.
- One small store, one place to edit it. A ProfileProvider beside the existing SettingsProvider, edited from the existing Display drawer. No accounts, no new routes, no backend change today.
- No name, no gender field, no device ID. English copy speaks to 'you'; Urdu and Pashto messages addressed to the learner come later, only as native-reviewed templates with a neutral default.
- Honest about privacy: your setup and history stay in this browser; pages you upload and quiz answers are processed on the server, as today.

2. The learner profile

Where it lives: New frontend/src/lib/profile.jsx (ProfileProvider / useProfile), a copy of the settings.jsx pattern: load() merges saved JSON over DEFAULTS inside try/catch, every change is saved inside try/catch, localStorage key 'roshni-profile-v1', mounted one line inside <SettingsProvider> in App.jsx. Kept separate from 'roshni-display-v1' so 'Reset to defaults' never wipes progress. Never sent to the server. Quiz settings move into frontend/src/lib/practiceConfig.js (same 'roshni-practice-v1' key) so a setup can update them.

Field	Values	How it is collected	What it changes	Default
onboarded	true / false	Set when the learner finishes or skips setup	Whether Home shows the 'Set Roshni up for you' card	false

needs	any combination of: reading_tiring, large_text, screen_reader, captions, second_language	Setup question 2 (multi-select chips), editable in Display	Applies small, additive setting changes that each chip literally promises, and picks the preferred tab	none (today's defaults)
preferredTab	read / listen / captions	Derived from needs by priority (screen_reader > captions > reading)	Lesson opens on this tab when the link has no ?tab. 'captions' is used only when the lesson has a lecture, otherwise Read Easy	read
levels	{ lessonId: { level: easier|simplest, from: quiz|manual, score, at } }	Recorded when the learner switches level, or when a quiz recommends a level	Read Easy opens at the remembered level, only if that version already exists; otherwise offers a button to make it	{}
recent	up to 8 { id, title, tab, at }, newest first	Recorded when a lesson is opened in this browser	Home 'Continue' card	[]
lastReport	{ lessonId, title, correct, total, level, weak headings, first plan step, at } or none	Saved from the quiz results in this browser	Home 'Your next step' card and the coach note's comparison with last time	none
best	{ lessonId: best score }	Updated after each quiz in this browser	Any 'your score' sentence (never read from the shared server history)	{}

3. Onboarding: "Tell Roshni about you"

Step	Question	Options	What it sets
1. Explain hard words in	Which language helps you most? (shown in large native script first)	اردو (Urdu) · پښتو (Pashto) · English only	settings.helpLang (already exists)
2. What helps you learn?	Choose any that fit you. You can change this any time under Display.	Reading is slow or tiring · I need large, clear text · I use a screen reader or keyboard only · I follow lessons by captions · English is not my first language · None of these	needs, preferredTab, and a few settings per chip (see the feature 'Two-question setup')

Skipping: A slim card at the top of Home: 'Set Roshni up for you · about 20 seconds' with 'Set up' and 'Not now'. Nothing opens by itself. 'Not now' hides the card and keeps today's defaults exactly; the same setup stays one tap away in the Display drawer.

Editing later: The existing Display drawer gets a 'Set up for me' section with the same two questions, plus 'Clear my data' beside 'Reset to defaults'.

Accessibility of onboarding: Built from native <fieldset>/<legend> and checkbox/radio inputs (like the font picker in DisplaySettings), so screen readers and keyboards work without new widgets. Plain short sentences,

large touch targets, native-script labels with lang/dir set, reduced motion respected, focus moves to the step heading, and results are announced through the existing live region.

4. The personal home

Section	What it shows	When
Set Roshni up for you	Slim card with 'Set up' and 'Not now'	Until setup is done or skipped
Continue	The last lesson opened in this browser, with its tab and a relative time	After the first lesson is opened
Your next step	The first study-plan step from your last quiz, labelled 'From your last quiz', opening the exact place	After the first quiz
Not you? Clear my data	Small link under the two cards	Whenever the For-you cards are shown
Everything else	Hero, upload, library and Fact Guard band stay as they are	Always

5. Features, in build order

Today (7 h)

Today **Learner profile store + Clear my data** 1 h

What the learner sees. Nothing directly; everything personal depends on it. In the Display drawer, 'Clear my data' appears beside 'Reset to defaults'.

How it works. lib/profile.jsx copies the settings.jsx pattern with its own key. Clear my data removes roshni-profile-v1, roshni-display-v1 and roshni-practice-v1 inside try/catch, then reloads the page so no provider writes the defaults back.

AI does: None.

Plain code does: Stores and loads the profile safely; works in private mode (falls back to defaults).

Data used: Only what this browser did: setup answers, lessons opened, levels, quiz results.

Gender and inclusivity: No name, gender or device ID is stored.

Accessibility: Clear my data asks for confirmation with a native button and announces the result.

Backend changes

None.

Frontend changes

- new lib/profile.jsx (ProfileProvider, useProfile)
- App.jsx: mount <ProfileProvider> inside <SettingsProvider>
- components/DisplaySettings.jsx: 'Clear my data' button

Risks: settings.reset() writes defaults back; mitigated by reload after removing keys

> **Done when:** Fresh browser: profile is the defaults. Private mode: no errors. Clear my data returns the app to a first-visit state.

Today **Any-page wording pass** 0.5 h

What the learner sees. Home and the upload card say Roshni works for any page you need to learn from: a textbook, a training manual, a safety notice, a worksheet, a medicine leaflet, an Islamiat or Urdu reader page, a form. Mode descriptions talk about needs ('If reading is slow or tiring') instead of diagnoses, and 'students' becomes 'learners'.

How it works. One static copy edit in Home.jsx and UploadPanel.jsx. Fact Guard and accessibility wording stay the same.

AI does: None.

Plain code does: Static text.

Data used: None.

Gender and inclusivity: Examples name objects, not roles, so nothing implies who does which job.

Accessibility: Shorter sentences at an easy reading level.

Backend changes

None.

Frontend changes

- pages/Home.jsx hero paragraph and MODES 'who' lines
- components/UploadPanel.jsx headline

Risks: Judges see the hero first; keep the Fact Guard sentence untouched

> **Done when:** Home mentions school, work, everyday and faith pages; no role nouns; Fact Guard band unchanged.

Today **Opens the way you need it, and says why** 2 h

What the learner sees. A lesson opens on your preferred tab and at the level that suits you, with one quiet line under the title: 'Opened at Simplest because your last quiz here was 4 of 10. Switch to Easier' or 'Opened in Listen because you said you use a screen reader. Change'. If the recommended version doesn't exist yet, a button offers 'Make the Simplest version for you', and AI writes it while Fact Guard checks it.

How it works. Order of priority: a study-plan step > the link's ?tab/&level > the profile > today's default. The preferred tab is lesson-aware ('captions' only when the lesson has a lecture). The remembered level is applied only when lesson.simplified[level] already exists, so opening a page never triggers an AI call. Choosing a level by hand is remembered for that lesson. Quiz settings move to lib/practiceConfig.js so a setup can set them, and Practice shows the level of the quiz actually returned (in demo mode, samples have Easier quizzes).

AI does: On request only: when the learner taps 'Make the Simplest version for you', the existing rewrite + Fact Guard pipeline runs. The AI does the work the personalization asks for.

Plain code does: Chooses tab and level, writes the 'because' line from stored data, and guards against AI calls on page load.

Data used: profile.preferredTab, profile.levels, lesson.simplified, lesson.lecture_id, the latest quiz result in this browser.

Gender and inclusivity: English second person only; the line is shown in English for now (Urdu/Pashto versions come with reviewed templates).

Accessibility: The line is announced once through the live region and can be dismissed for that lesson. For screen-reader users, Listen keeps the full text (tables and equations are not dropped) and voice quiz is never switched on automatically.

Backend changes

None.

Frontend changes

- pages/Lesson.jsx: initial tab from profile (lesson-aware), seed focus from route params once, then strip them
- modes/ReadEasy.jsx: initial level from profile with the cache guard, remember manual choices, offer button
- new components/BecauseLine.jsx
- new lib/practiceConfig.js; modes/Practice.jsx reads it and shows quiz.level
- App.jsx: pass route.params to Lesson

Risks: ReadEasy currently simplifies an uncached level on mount; the guard is required; A reloaded link must not override a manual level choice

> **Done when:** With the screen-reader setup, a sample opens in Listen with the 'because' line; after a quiz under 50%, the same page opens at Simplest with the reason; no simplify request is made on page load (Network tab).

Today **Home 'For you': Continue and Your next step** 1.25 h

What the learner sees. Returning learners see two cards above the library: 'Continue: 5.1 Overview of Photosynthesis · Listen · yesterday' and 'Your next step: Re-read “Solar Dependence and Food Production” at the Simplest level · From your last quiz' with the button opening exactly that section (or word, or video minute). A small 'Not you? Clear my data' link sits underneath. First-time visitors see nothing extra.

How it works. Continue comes from profile.recent; Next step is the first step of the study plan saved from this browser's last quiz (quiz.study_plan already produces deep-linkable actions). The link carries ?tab=&level=&section=&term=&video=&t= and Lesson seeds its focus from them once.

AI does: None on Home; the step points to AI-made, Fact-Guarded content.

Plain code does: Builds both cards from the local profile only (never from the shared server history, which mixes all visitors' scores).

Data used: profile.recent, profile.lastReport.

Gender and inclusivity: Neutral English; no greeting by name.

Accessibility: Real links in reading order, clear accessible names, relative times written in words, no autoplaying speech.

Backend changes

None.

Frontend changes

- new components/ForYou.jsx
- pages/Home.jsx: render ForYou above the library
- pages/Lesson.jsx: record recent on open
- modes/Practice.jsx: save lastReport and best after results

Risks: A lesson deleted on the server: hide its card when it's missing from /api/lessons

> **Done when:** After opening two lessons and finishing a quiz under 70%, Home shows both cards, and Next step opens the weak section at Simplest within one click.

Today **Coach note on results (plain code)** 0.5 h

What the learner sees. Under the score, one or two sentences written for this attempt: 'Up from 4 to 7 out of 10 since last time. “Solar Dependence and Food Production” is still the part to practise: a heterotroph is an organism that eats other organisms.'

How it works. lib/coach.js builds the note from the current report and the previous attempt on the same lesson in this browser: the change in score, a section that is weak again, and one missed fact quoted word for word from the verified facts. It replaces the generic advice line, and is spoken only when read-aloud is on.

AI does: None today: this is the permanent fallback for the checked AI coach planned after the hackathon.

Plain code does: Every number and fact in the note is copied from the report; nothing is invented.

Data used: The quiz report and profile.lastReport for the same lesson.

Gender and inclusivity: Second person, neutral, no names.

Accessibility: Announced once in the live region; capped at two short sentences.

Backend changes

None.

Frontend changes

- new lib/coach.js
- modes/Practice.jsx results card uses it

Risks: First attempt has no comparison: show only the missed-fact sentence

> **Done when:** Two attempts on one lesson produce a note whose numbers match both result cards exactly.

Today **Two-question setup (card on Home + Display drawer)** 1.75 h

What the learner sees. A slim 'Set Roshni up for you · about 20 seconds' card on Home. 'Set up' shows two questions: which language explains hard words (پښتو / اردو / English only), and what helps you learn (choose any). Choices take effect at once, and the same section lives in the Display drawer.

How it works. Each chip adds only what it promises. Reading is slow or tiring: Lexend font, wider spacing, Simplest level, timer off. Large text: bigger text and line spacing. Screen reader or keyboard only: Listen first, timers off, quiz read-aloud off (the screen reader already reads), tap mode kept. Captions: Captions first when a lecture exists, read-aloud off. English is not my first language: Simplest level and word explanations in the chosen language. Chips combine.

AI does: None in setup; the choices decide which AI-made versions you get.

Plain code does: One PRESETS map applied to settings, profile and practiceConfig.

Data used: The learner's own answers.

Gender and inclusivity: No name, gender or 'how to address you' question. Chips describe what helps, not diagnoses.

Accessibility: Native fieldset/checkbox inputs, native-script labels with lang/dir, short reviewed Urdu glosses under chips if an Urdu reader on the team can check them tonight.

Backend changes

None.

Frontend changes

- new components/SetupCard.jsx and components/SetupQuestions.jsx
- components/DisplaySettings.jsx: 'Set up for me' section
- pages/Home.jsx: card when not onboarded

Risks: Presets must not force a microphone prompt or an overlay nobody asked for

> **Done when:** Keyboard only, with a screen reader: setup completes in under a minute, and the next lesson opens as promised.

Demo polish (1 h)

Demo polish **Keyboard fixes in shared UI parts** 0.75 h

What the learner sees. Arrow keys move between options in every switch (level, timer, language), and the Display drawer keeps keyboard focus inside it until closed.

How it works. Roving tabindex with arrow keys in Segmented; a Tab/Shift+Tab focus trap in Drawer, with first focus on the drawer heading.

AI does: None.

Plain code does: Accessibility fixes in components/ui.jsx.

Data used: None.

Gender and inclusivity: Not applicable.

Accessibility: Fixes keyboard use for the existing Display drawer and Practice setup too.

Backend changes

None.

Frontend changes

- components/ui.jsx: Segmented onKeyDown + roving tabindex; Drawer focus trap

Risks: Touches shared components; re-test Read Easy level switch and Practice setup

> **Done when:** Tab never leaves the open drawer; arrow keys change every segmented control.

Demo polish **Demo-ready profile on the final site** 0.25 h

What the learner sees. On the demo laptop, Home already shows Continue and Your next step, made by actually using the deployed app.

How it works. After the final deploy, run setup, open two samples and finish one quiz under 70% on the deployed address (browser storage belongs to each address). Sunday morning is rehearsal only, no new code.

AI does: None.

Plain code does: None.

Data used: Real use the evening before.

Gender and inclusivity: Not applicable.

Accessibility: Not applicable.

Backend changes

None.

Frontend changes

None.

Risks: Seeding before the final deploy produces an empty strip on stage

> **Done when:** Fresh start of the demo browser shows both cards on the deployed link.

After the hackathon (17 h)

After the hackathon

Review what you missed

4 h

What the learner sees. When you have missed facts waiting: '3 facts to review' on Home, opening a short quiz on just those facts. A fact leaves the list once you answer it correctly.

How it works. Missed fact ids saved per attempt (quiz questions already carry fact_id; add fact_id to the check response). Questions come from the cached question bank first; AI is called only for facts with no cached question.

AI does: Writes questions for facts that have none, from verified facts only.

Plain code does: Keeps the list, picks due facts, validates questions as today.

Data used: Missed fact ids in this browser.

Gender and inclusivity: Neutral.

Accessibility: Works with tap, keys and voice like Practice.

Backend changes

- quiz.check_one returns fact_id
- make_quiz accepts fact_ids

Frontend changes

- profile.review list
- Home card when the list is not empty

Risks: Keep it to one simple rule, no streaks or daily targets

> **Done when:** A missed fact appears, is asked, and disappears after a correct answer.

After the hackathon

Ask about this sentence (grounded tutor)

4 h

What the learner sees. Tap a sentence and ask a question in English, Urdu or Pashto. The answer quotes the page, or says 'The page doesn't say that'.

How it works. The model sees only the section containing the sentence. Every answer sentence must carry a quote that passes factguard.evidence_ok, and numbers must appear in the quote. Limited per lesson per hour.

AI does: Answers the learner's own question, something plain code cannot do.

Plain code does: Quote and number checks, limits, 'the page doesn't say' fallback.

Data used: The section text and the learner's question.

Gender and inclusivity: Third person in Urdu/Pashto answers until reviewed templates exist.

Accessibility: Works with voice input (Whisper) and read-aloud.

Backend changes

- POST /api/lessons/{id}/ask
- prompts/ask.txt

Frontend changes

- Ask button on sentences in Read Easy and Listen

Risks: Free-tier quota; cap requests

> **Done when:** Questions outside the page get 'The page doesn't say that'; every answer shows its quote.

After the hackathon

Checked AI coach (v2)

3 h

What the learner sees. A slightly richer note that links your miss to the exact line on the page, with a 'Checked' pill.

How it works. Served after results appear, from a separate endpoint. Code writes every number; the model may write no digits; the next move must copy a study-plan title exactly; the missed fact is inserted by code; cached on the exact result. Falls back to the plain-code note when AI is busy.

AI does: Phrases a short explanation connecting the miss to the page.

Plain code does: Supplies all facts and numbers and rejects any sentence that breaks the rules.

Data used: Score, level, missed fact ids (no name, no profile).

Gender and inclusivity: Neutral English; Urdu/Pashto only through reviewed templates.

Accessibility: Same as the coach note.

Backend changes

- POST /api/quiz/{id}/coach
- prompts/coach.txt

Frontend changes

- Results card fetches coach v2

Risks: Must never block the results screen

> **Done when:** No coach sentence contains a number that isn't in the report.

After the hackathon

Urdu and Pashto messages with your chosen form of address

3 h

What the learner sees. Results and next steps in Urdu or Pashto. Display adds 'How should Roshni address you in Urdu or Pashto?', with each option shown as a sample sentence and a neutral default.

How it works. Fixed templates signed off by native speakers. Neutral Urdu uses only آپ with imperatives, present/future with invariable words, object-agreeing perfective ('آپ نے 10 میں سے 6 درست کیے') or nominal sentences; neutral Pashto uses تاسو with imperatives, present tense, object-agreeing perfective or nominal lines. A message that can't be neutral is left out rather than forced.

AI does: None: templates, not generation.

Plain code does: Fills templates with numbers and headings.

Data used: addressForm (learner-chosen, optional).

Gender and inclusivity: The learner chooses; never inferred; neutral is the default.

Accessibility: Spoken with the existing Urdu/Pashto voices.

Backend changes

None.

Frontend changes

- lib/messages.js templates
- Display: address-form choice

Risks: Unreviewed grammar; ship only with native sign-off

> **Done when:** A native Urdu and a native Pashto reader approve every line in all forms.

After the hackathon **Private progress on shared servers + choice of voice** 3 h

What the learner sees. On a hosted site, progress shows only your own attempts; Display lets you pick the Urdu voice (Uzma or Asad) and the Pashto voice (Latifa or Gul Nawaz), labelled by name with a sample.

How it works. Quiz attempts stored under an anonymous per-browser key made only when needed; TTS accepts an allow-listed voice.

AI does: None.

Plain code does: Storage and allow-list.

Data used: An anonymous random key.

Gender and inclusivity: Voices are a preference, not a gender question.

Accessibility: Pick the voice you understand best.

Backend changes

- practice history keyed per browser
- tts voice parameter

Frontend changes

- Display voice picker

Risks: Pre-make sample audio for demo mode

> **Done when:** Two browsers on one server see only their own history.

6. Where AI is a real layer, and how each use is checked

Where	Model	What it adds	How it is checked / behaves without AI
Read Easy: the version made for your level	Qwen 3.8 27B, checked by GPT-OSS-120B (Fact Guard)	When your quiz or setup calls for Simplest, AI writes that version for you	Fact Guard audit + number check; only on request, never on page load; demo mode serves cached versions
Hard words in your language	Gemini (Qwen backup)	Explanations in Urdu or Pashto, chosen in setup	Cached per lesson; terms picked by plain code
Practice questions	Gemini (Qwen backup)	Questions from the facts on your page, at your wording level	Only verified facts; 4 options, one correct, validated in code
Voice answers	Whisper large-v3-turbo	Answer out loud in English, Urdu or Pashto	Plain code decides the option; unclear replies are asked again
Video notes	Qwen, checked by GPT-OSS-120B	Notes from any YouTube video you learn from	Every note quoted from the captions
After the hackathon: Ask about this sentence	Free-tier text model	Answers your own questions about the page	Quotes must pass evidence_ok; 'The page doesn't say that' fallback
After the hackathon: checked coach v2	Free-tier text model	Connects your miss to the exact line	No digits from the model; plan titles copied exactly; plain-code note as fallback

7. Rules we hold ourselves to

Friendly for all genders

- No name field, no gender field, no avatar, no gendered colours or icons. 'Welcome back' at most; never greet by name.
- Never infer gender from anything (names, voice, subjects, choices).
- English copy is second person ('you') everywhere; no he/she, his/her.
- Examples name objects (a training manual, a recipe, a medicine leaflet), never roles that suggest who does which job.
- Urdu and Pashto messages addressed to the learner ship only as native-reviewed templates with a learner-chosen form of address and a truly neutral default (see A4 for the grammar rules); until then they are not shown.
- Voice choice, when it comes, is labelled by name with a sample, as a preference, never as 'male/female'.
- Setup chips describe what helps, not diagnoses or identities.

Any kind of learning, every day

- Home says 'any page you need to learn from' and lists school, work, everyday and faith pages.
- Nothing in setup asks for a goal or a career; the same flow serves an exam student, a trainee and someone reading a recipe or a reader for Islamiat.
- Videos and Practice work for any topic with a page or captions.
- Continue and Your next step make Roshni the place you come back to every day, for whatever you're learning.

Don't overdo it

- At most two setup questions, and nothing opens by itself.
- At most two For-you cards on Home.
- No streaks, badges, points, confetti or 'days this week'.
- No personal sentence without stored data behind it.
- No new AI call on page load; AI only when the learner asks or acts.
- Every tailored default is explained in one line and can be changed in one tap.
- Nothing new is spoken automatically; the screen reader or read-aloud setting decides.

Privacy

- Your setup and your progress stay in this browser (localStorage). No account, name, gender or device ID.
- Pages you upload and quiz answers are processed and stored on our server, as today; we say so plainly.
- 'Clear my data' in Display (and 'Not you?' on Home) removes everything this browser stored.
- Nothing from the profile is sent to AI providers.
- After the hackathon: per-browser quiz history on shared servers, so other visitors' scores never mix with yours.

8. Rollout

Today, in order (7 h total):

1. P1 Learner profile store + Clear my data (1 h)
2. P2 Any-page wording pass (0.5 h)
3. P3 Opens the way you need it, with the 'because' line (2 h)
4. P4 Home 'For you': Continue + Your next step (1.25 h)
5. — Cut line: P1–P4 alone deliver the full 'set up for me' story (4.75 h) —
6. P5 Coach note on results, plain code (0.5 h)
7. P6 Two-question setup card + Display section (1.75 h)
8. Test: demo mode and live mode, keyboard + screen reader pass, backend tests, then deploy and seed the demo profile (D2)

> **The demo moment:** About 20 seconds inside the live demo: open Home on the deployed site (profile seeded the evening before). Continue and Your next step are already there. Click Your next step: the weak section opens at Simplest with 'Opened at Simplest because your last quiz here was 4 of 10'. Say: 'Roshni remembers how this learner reads and what they missed. Plain code picks the next step; the AI wrote the simpler version, and Fact Guard checked it.'

After the hackathon, in order:

1. D1 Keyboard fixes in shared UI parts (if not done tonight)
2. A1 Review what you missed
3. A2 Ask about this sentence (grounded tutor)
4. A3 Checked AI coach v2
5. A4 Urdu and Pashto messages with a chosen form of address (native review)
6. A5 Private progress on shared servers + choice of voice

9. How we will know it works

- In a fresh browser, the demo path (setup → lesson opens as promised → quiz → Home shows the next step) takes under 60 seconds.
- Opening any lesson makes no simplify or quiz request unless the learner asks (checked in the Network tab).
- In demo mode, Home and lessons work with no AI calls.
- Every personal sentence on screen can be traced to a stored value (code review).
- Keyboard-only and screen-reader users can complete setup and reach their next step.
- After the hackathon: a pilot with real learners measures whether personalized defaults reduce time to the right mode.

10. What we are deliberately not doing

Idea	Why not
A setup pop-up that opens by itself on the first visit	It steals focus from screen readers and interrupts the first view; an inline card does the same job on request.
Name field and 'Welcome back, <name>'	Personal data with no learning value; leaks on shared phones and lab computers; invites gender assumptions.
Gender or form-of-address question now	Nothing uses it until reviewed Urdu/Pashto templates exist; it would be a gender question with no payoff.

Streaks, badges, points, 'days this week'	Gamification that nags; the brief says don't overdo it.
Personal scores from the server's lesson history	That history mixes every visitor's attempts on a shared server, so it would show strangers' scores as yours.
Interest or goal questionnaires and personalized prompts	Facts don't change with goals; it adds an injection risk, breaks the cache and costs free-tier quota.
Spaced repetition with boxes and daily minute targets	Can't be observed or tested before Sunday; a simple review list comes after the hackathon.
Speaking a Home summary automatically	Talks over screen readers and is blocked by browsers before a click.
Turning on voice quiz or simpler Listen for screen-reader users	Triggers an unasked microphone prompt and drops tables and equations from their audio.
An AI coach tonight	It needs strict number checks to be trustworthy; the plain-code note is honest today, and the checked version comes later.

11. Open questions for the team

- Who on the team can review short Urdu labels under the setup chips tonight (and Pashto, if anyone can)?
- When is the final deploy, so the demo profile can be seeded on that address afterwards?
- Do we build P5 and P6 tonight, or stop at the cut line and spend the time on the deck and rehearsal?
- Should the 'because' line also mention the Fact Guard count ('14 of 14 facts kept'), or keep it to one reason?

Produced from a design review: a codebase map, four independent proposals (learner-first, everyday platform, AI layer, minimalist), three critiques each (over-engineering and time, inclusivity and accessibility, AI value and privacy), two judges, one synthesis, and three verification passes. Effort estimates assume one developer working with an AI coding assistant.